

LuaLaTeX + jlreq ではじめる日本語 L^AT_EX 入門

kska6

2026-05-11

目次

1	この資料の目的	3
2	まず動かす	3
2.1	最小構成	3
2.2	コンパイル	3
2.3	hello.tex でエンジンを見る	4
3	LuaLaTeX と jlreq	4
3.1	なぜ LuaLaTeX を使うのか	4
3.2	なぜ jlreq を使うのか	4
4	プリアンブルと package	4
4.1	プリアンブルとは何か	5
4.2	この資料で使う package	5
4.3	不要な package を増やさない	5
5	文章の構造を書く	5
5.1	見出し	6
5.2	段落と改行	6
5.3	箇条書き	6
5.3.1	itemize	6
5.3.2	enumerate	6
5.3.3	ネスト	6
6	数式を書く	7
6.1	インライン数式	7
6.2	equation	7
6.3	align	7
6.4	aligned	8
6.5	cases と dcases	8
6.6	連立方程式	9
6.7	ベクトルと太字	9
6.8	数式番号と参照	9
7	図表を書く	10

7.1	図	10
7.2	表	10
7.3	配置指定と [H]	11
7.4	図表参照	11
8	ラベルと参照	11
8.1	基本	11
8.2	hyperref	12
8.3	zref	12
9	参考文献を書く	12
9.1	なぜ .bib を分けるのか	12
9.2	biblatex と biber	12
9.3	latexmk との関係	13
9.4	VSCoDe + LaTeX Workshop との関係	13
9.5	参考資料	13
10	latexmkrc とビルド設定	13
10.1	latexmkrc の役割	13
10.2	LuaLaTeX + biber の設定例	13
10.3	outdir を分ける理由	14
10.4	VSCoDe 設定との対応	14
10.5	VSCoDe settings.json の例	14
11	ファイル構成	15
11.1	最小構成と実運用構成の違い	15
11.2	推奨構成	15
11.3	.gitignore の例	15
11.4	Makefile の簡単な例	16
12	よくあるエラー	16
12.1	閉じ忘れ	16
12.2	全角記号の混入	16
12.3	package 読み込み順	16
12.4	補助ファイルを消すと直ることがある	16
13	避けるべき書き方	16
14	発展トピック	17
15	参考資料	17
16	まとめ	17

1 この資料の目的

この資料は、LuaLaTeX と jlreq を前提に、日本語のレポートや論文を書き始めるための入門資料です。目標は、次の 3 点です。

- まずコンパイルが通る最小構成を作れるようになる
- 数式、図表、参考文献、相互参照を段階的に追加できるようになる
- 卒業論文や技術文書へ拡張しやすい書き方を身につける

この資料では「何を書くとうなるか」だけでなく、「なぜその環境や記法を使うのか」も重視します。L^AT_EX では見た目を直接いじるより、文書の構造を明示して書く方が、長い文書になったときに強くなります。

2 まず動かす

2.1 最小構成

最初の目標は、機能を盛り込むことではなく、コンパイルを成功させることです。このリポジトリには、最小構成の例として `src/minimal.tex` を用意しています。

```
\documentclass{jlreq}

\title{LaTeX の最小構成}
\author{氏名}
\date{\today}

\begin{document}
\maketitle

\section{はじめに}

これは LuaLaTeX で作成する最小構成の文書です。

\section{数式の例}

文中の数式は  $E = mc^2$  のように書きます。

\begin{equation}
F = ma
\end{equation}

\end{document}
```

この段階では、`\documentclass`、`\begin{document}` と `\end{document}`、`\section`、`equation` だけ追えば十分です。最初から多くの package を入れるより、最小構成が通る状態を基準にした方が、問題が起きたときに切り分けやすくなります。

2.2 コンパイル

最小構成は、次のようにコンパイルできます。

```
latexmk -lualatex src/minimal.tex
```

lualatex は実際に PDF を作るエンジンで、latexmk は必要な回数だけ再実行したり、参考文献ツールを呼び出したりする管理役です。普段の作業では、lualatex を直接何度も叩くより、latexmk を入口にそろえる方が運用しやすくなります。

2.3 hello.tex でエンジンを見る

minimal.tex の次は、src/hello.tex を使って「いま何のエンジンで動いているか」を確認すると理解しやすくなります。hello.tex は文書の内容を増やすための例ではなく、コンパイル系統の違いを見るための小さなサンプルです。

```
latexmk -lualatex src/hello.tex
```

このファイルをビルドすると、出力には Hello, LuaLaTeX! のように、実際に使われたエンジン名が現れます。つまり、minimal.tex で「まず通す」、hello.tex で「何で通しているかを確認する」という二段構えにすると、入門時の導線が整理しやすくなります。

3 LuaLaTeX と jlreq

3.1 なぜ LuaLaTeX を使うのか

日本語文書では、エンジンと日本語処理の相性が重要です。LuaLaTeX は luatexja 系と組み合わせやすく、日本語を比較的素直に扱えます。入門段階では、「日本語レポートを通しやすい構成」として LuaLaTeX を選ぶ、と理解すれば十分です。

ここで hello.tex を見ると、「エンジン」とは .tex を処理して PDF へつなぐ実体であり、LuaLaTeX や XeLaTeX などの違いがあることを、抽象論だけでなく出力結果として確認できます。

3.2 なぜ jlreq を使うのか

jlreq は日本語組版を意識して設計された文書クラスです。英語向けの標準クラスを無理に流用するより、日本語の見出しや版面を自然に扱いやすくなります。レポート用途では既定の article 相当で始めれば十分で、長い文書では次のように切り替えます。

```
\documentclass[a4paper,report]{jlreq}  
\documentclass[a4paper,book]{jlreq}
```

report は章立てのある報告書向け、book はさらに長い冊子向けです。講義レポートなら既定設定、卒論や章構成を持つ文書なら report をまず検討すると整理しやすくなります。

4 プリアンプルと package

4.1 プリアンプルとは何か

`\begin{document}` より前に書く設定部分をプリアンプルと呼びます。ここには、文書全体で使う package、ページ設定、番号付け、独自コマンドなどを書きます。本文の途中で毎回同じ設定を書くのではなく、共通設定を一箇所にまとめるための領域だと考えるとよいです。

4.2 この資料で使う package

この資料の冒頭では、次のように package を読み込んでいます。

```
% Japanese text setup for LuaLaTeX. Keep this near the top.
\usepackage[haranoaji]{luatexja-preset}

% Page layout. Reports are easier to read when margins are explicit.
\usepackage[margin=25truemm]{geometry}

% Math basics. amsmath/mathtools are the core pair for report writing.
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{mathtools}
\usepackage{bm}

% Figures and tables. float is for the optional [H] placement.
\usepackage{graphicx}
\usepackage{booktabs}

% Bibliography and hyperlinks. hyperref stays near the end.
\usepackage[backend=biber,style=numeric,sorting=nyt]{biblatex}
\usepackage{hyperref}
```

読み込み順にも理由があります。luatexja-preset は日本語環境の基盤なので早めに置き、hyperref は他の機能にリンクを付ける都合で後ろ側に置くのが定石です。mathtools は amsmath を拡張する package なので、amsmath の後に読みます。

4.3 不要な package を増やさない

package は便利ですが、増やしすぎると依存関係や競合が見えにくくなります。特に入門段階では、次の順で考えると安全です。

1. まず最小構成で通す
2. 必要な機能が出たときに package を 1 つずつ追加する
3. 追加理由をコメントで残す

たとえば、図を入れない文書に graphicx は不要です。参考文献をまだ扱わない段階で biblatex を入れなくても構いません。「今この文書に何の機能が必要か」を基準にすると、保守しやすいプリアンプルになります。

5 文章の構造を書く

5.1 見出し

L^AT_EX では、見た目を大きくするより、見出しとしての役割を命令で示します。

```
\section{実験方法}
\subsection{装置}
\subsection{手順}
```

この書き方にすると、見出し番号や目次との連携が自動化されます。後から節を追加しても番号を手で直す必要がありません。

5.2 段落と改行

空行は段落の区切り、\\ は行の折り返しです。文章を書くときは、見た目をそろえるために \\ を多用するのではなく、段落構造で整理します。\\ は住所や詩のように、意味として改行したい場合に限って使う方が安全です。

5.3 箇条書き

箇条書きは、文章を構造化して読みやすくするために使います。単に行頭へ記号を並べるのではなく、項目のまとまりを環境として表します。

5.3.1 itemize

itemize は順序を持たない列挙です。

```
\begin{itemize}
  \item 目的を書く
  \item 方法を書く
  \item 結果を書く
\end{itemize}
```

項目ごとに \item を書くと、字下げや記号の配置が自動で整います。

5.3.2 enumerate

enumerate は順序を持つ列挙です。手順や実験プロセスの説明でよく使います。

```
\begin{enumerate}
  \item データを整理する
  \item グラフを作成する
  \item 考察を書く
\end{enumerate}
```

5.3.3 ネスト

箇条書きは入れ子にできます。ただし深くしすぎると読みにくくなるため、二段程度までに抑える方が実用的です。

```
\begin{itemize}
  \item レポート本文
  \begin{enumerate}
    \item はじめに
    \item 実験方法
    \item 結果と考察
  \end{enumerate}
  \item 参考文献
\end{itemize}
```

6 数式を書く

6.1 インライン数式

文中に短い数式を入れるときは、 $\$...\$$ を使います。

質量 m の物体に力 F が加わるとき、加速度は $a = F/m$ です。

本文の流れを切らずに記号を説明したいときに向いています。逆に、式変形の途中や参照したい式までインラインで詰め込むと読みにくくなります。

6.2 equation

`equation` は、独立した 1 本の式を番号付きで示したいときに使います。

```
\begin{equation}
E = mc^2
\label{eq:energy}
\end{equation}
```

本文中から (6.1) のように参照したいなら、`equation` を使う価値があります。独立した式に番号を持たせることで、後から説明や考察で再利用しやすくなるためです。

$$F = ma \tag{6.1}$$

6.3 align

`align` は、複数行の式をそろえて書きたいときに使います。等号の位置を合わせると、変形の流れが読み取りやすくなります。

```
\begin{align}
y &= (x + 1)^2 \\
&= x^2 + 2x + 1
\end{align}
```

$$y = (x + 1)^2 \quad (6.2)$$

$$= x^2 + 2x + 1 \quad (6.3)$$

`equation` と `align` の違いは、主に「1 行を独立して見せたいか」「複数行の対応関係を見せたいか」です。1 本の式なら `equation`、変形を追わせたいなら `align` を使います。

6.4 aligned

`aligned` は単独では使わず、通常は `equation` などの中に入れて使います。「式全体としては 1 番号にしたいが、中では複数行をそろえたい」ときに便利です。

```
\begin{equation}
\begin{aligned}
f(x)
&= (x + 1)(x - 1) \\
&= x^2 - 1
\end{aligned}
\label{eq:aligned-example}
\end{equation}
```

$$\begin{aligned} f(x) &= (x + 1)(x - 1) \\ &= x^2 - 1 \end{aligned} \quad (6.4)$$

`align` は各行を並べるための外側の環境、`aligned` は他の数式環境の中で行揃えだけをしたいときの内側の環境、と覚えると区別しやすくなります。

6.5 cases と dcases

場合分けを書くときは `cases` を使います。

```
\begin{equation}
|x| =
\begin{cases}
x & (x \geq 0) \\
-x & (x < 0)
\end{cases}
\end{equation}
```

$$|x| = \begin{cases} x & (x \geq 0) \\ -x & (x < 0) \end{cases} \quad (6.5)$$

`cases` の各行は本文スタイルで組まれるため、分数などが少し小さく見えることがあります。そこで `mathtools` の `dcases` を使うと、各行が `displaystyle` で組まれ、式が見やすくなる場合があります。

```
\begin{equation}
f(n) =
\begin{dcases}
```



```
\dfrac{n}{2} & (n \text{ が偶数}) \\
\dfrac{3n + 1}{2} & (n \text{ が奇数}) \\
\end{dcases}
\end{equation}
```

$$f(n) = \begin{cases} \frac{n}{2} & (n \text{ が偶数}) \\ \frac{3n+1}{2} & (n \text{ が奇数}) \end{cases} \quad (6.6)$$

短い式だけなら `cases` で十分ですが、分数や総和のように大きめの記号を含む場合は `dcases` の方が読みやすいことがあります。

6.6 連立方程式

連立方程式では、左側の大きな波括弧と、各式の揃えを同時に扱いたくなります。このとき `aligned` を使うと、「全体は 1 個の数式」として扱いながら中だけ整列できます。

```
\begin{equation}
\begin{cases}
\begin{aligned}
2x + y &= 1 \\
x - y &= 3
\end{aligned}
\end{cases}
\end{equation}
```

$$\begin{cases} 2x + y = 1 \\ x - y = 3 \end{cases} \quad (6.7)$$

連立方程式で `aligned` を使う理由は、各行の等号位置をそろえつつ、式全体の番号を 1 つにまとめるからです。独立した複数の式ではなく、1 組の方程式として見せたい場面向いています。

6.7 ベクトルと太字

数式中では、ベクトルや行列を太字で表したいことがあります。`\boldsymbol` でも書けますが、`\bm` を入れておくと `\bm` で簡潔に書けます。

```
\begin{equation}
\bm{F} = m\bm{a}
\end{equation}
```

$$\boldsymbol{F} = m\boldsymbol{a} \quad (6.8)$$

6.8 数式番号と参照

数式番号を本文で手打ちしてはいけません。番号は編集のたびにずれるので、式の直後に `\label` を置き、本文では `\eqref` で参照します。

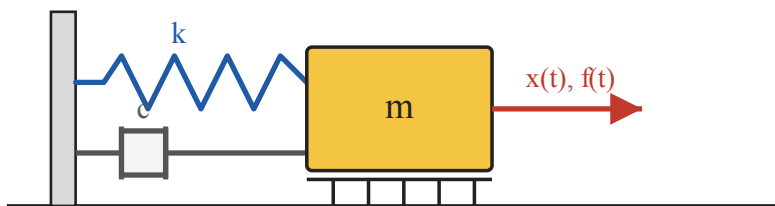


図 1 質量 - バネ - ダンパ系

```
\begin{equation}
F = ma
\label{eq:newton}
\end{equation}
```

式 `\eqref{eq:newton}` より、力と加速度は比例します。

`\ref` は一般の番号参照、`\eqref` は数式参照で丸括弧まで含めて出したいときに使います。式番号を「(3.1)」のように直接打つより、修正に強い書き方です。

7 図表を書く

7.1 図

画像を入れるには `graphicx` と `\includegraphics` を使います。図番号を手打ちするのではなく、`\caption` と `\label` を組み合わせて管理します。

```
\begin{figure}[tb]
\centering
\includegraphics[width=.7\linewidth]{../fig/mass-spring-damper.pdf}
\caption{質量--ばね--ダンパ系}
\label{fig:mass-spring-damper}
\end{figure}
```

`\label` は `\caption` の後に置きます。図番号は `\caption` が進めるため、その前に `\label` を置くと意図しない番号を拾うことがあります。また、画像ファイルは `fig/` に分けておくと、本文ソースと素材を整理しやすくなります。

7.2 表

表では `table` が浮動体、`tabular` が中身の表組です。役割が違うので、両方の役目を分けて理解すると混乱しにくくなります。

```
\begin{table}[tb]
\centering
\caption{モデルのパラメータ}
\label{tab:parameters}
\begin{tabular}{lll}
```

表 1 モデルのパラメータ

記号	意味	単位
m	質量	kg
k	ばね定数	N/m
c	減衰係数	N s/m

```

\toprule
記号 & 意味 & 単位 \\
\midrule
 $m$  & 質量 & kg \\
 $k$  & ばね定数 & N/m \\
 $c$  & 減衰係数 & N s/m \\
\bottomrule
\end{tabular}
\end{table}

```

booktabs を使うと、`\toprule`、`\midrule`、`\bottomrule` で見やすい横罫線を書けます。実務や論文では、縦罫線を増やすより、余白と横罫線で整理する方が読みやすいことが多くあります。

7.3 配置指定と [H]

`figure` や `table` は浮動体なので、ソースに書いた位置へ必ずそのまま出るとは限りません。通常は `[tb]` や `[htbp]` のように候補位置を与えて、配置は L^AT_EX にある程度任せる方が紙面を崩しにくくなります。

どうしても「この位置にそのまま置きたい」ときだけ、`float` の `[H]` を使います。

```

\begin{figure}[H]
\centering
\includegraphics[width=.7\linewidth]{../fig/mass-spring-damper.pdf}
\caption{この位置に固定したい図}
\label{fig:fixed-example}
\end{figure}

```

`[H]` は便利ですが、使いすぎると図表が詰まり、かえって読みづらくなることがあります。まずは通常の浮動配置で様子を見て、本当に固定が必要な図表だけに絞る方が実運用では安全です。

7.4 図表参照

図や表も番号を手打ちせず、`\ref` を使います。たとえば、図 1 や表 1 のように参照できます。これにより、図表を追加して番号がずれても本文を手で追い直す必要がありません。

8 ラベルと参照

8.1 基本

`\label` は「この場所に名前を付ける」命令、`\ref` は「その番号を呼び出す」命令です。相互参照の基本は次の形です。

```
\section{結果}
\label{sec:result}

第\ref{sec:result}節で結果を述べる。
```

式なら `\eqref`、図表や節なら `\ref` を使う、という整理で十分です。いずれの場合も、番号そのものを本文へ打ち込まないことが重要です。

8.2 hyperref

`hyperref` を入れると、目次、参照、URL が PDF 上でクリック可能になります。長いレポートや論文では、参照先へ飛べるだけで読みやすさが大きく変わります。そのため、入門段階でも相互参照を使う文書では導入しておく価値があります。

8.3 zref

`zref` は、相互参照をより柔軟に扱いたいときに使う発展的な package です。基本的なレポートなら `\label` と `\ref`、`\eqref`、そして `hyperref` で十分ですが、参照に付ける情報を増やしたいときに `zref` が候補になります。

たとえば、次のような場面で検討できます。

- 図、表、式、節の参照表記をまとめて整えたい
- 番号だけでなくページ情報なども扱いたい
- 複数種類の参照ルールを文書全体で統一したい

`zref` 自体は基盤寄りの package で、実際には `zref-clever` のような拡張と組み合わせて使うことがあります。ただし、初学者向けの最初の 1 本では導入対象を増やしすぎない方が安全です。まずは標準の参照を使い、文書が大きくなって参照表記の統一が負担になってきた段階で、`zref` 系を検討するとよいです。

9 参考文献を書く

9.1 なぜ .bib を分けるのか

参考文献を本文中へ直接並べることもできますが、実運用では `.bib` ファイルへ分ける方が管理しやすくなります。文献情報を本文から分離すると、複数文書で再利用しやすく、書誌情報の修正も一箇所で済みます。

このリポジトリでは `bib/references.bib` を用意しており、本文側では次のように読み込みます。

```
\usepackage[backend=biber,style=numeric]{biblatex}
\addbibresource{../bib/references.bib}
```

9.2 biblatex と biber

biblatex は本文側のインターフェース、biber は参考文献データの処理役です。役割を分けて考えると理解しやすくなります。

- biblatex: `\cite` や `\printbibliography` など、L^AT_EX 側の命令を提供する
- biber: `.bib` を読み、引用や文献一覧を整形するための補助処理を行う

本文では [3] のように引用し、文末で `\printbibliography` を呼びます。biblatex までは入れたのに biber が実行されていないと、引用が未解決のまま残ります。

9.3 latexmk との関係

latexmk を使う利点は、biber の実行も含めて流れを自動化できることです。参考文献を使う文書では、`lualatex` 1 回では足りません。latexmk を入口にしておくで、「biber を忘れた」という初学者のつまずきを減らせます。

9.4 VSCode + LaTeX Workshop との関係

VSCode の LaTeX Workshop でも、実体としては latexmk を呼ぶ設定にしておくで、ターミナル実行とエディタ実行の差が小さくなります。エディタ専用の複雑な魔法を作るより、コマンドラインで通る設定をそのまま使う方が再現性が高くなります。

9.5 参考資料

biblatex の詳細な運用は公式マニュアルや実運用例も合わせて確認するとよいですが、入門段階では機能を一度に追いつぎず、まずは `\addbibresource`、`\cite`、`\printbibliography` の 3 つを基準に使い始めるのが安全です。

10 latexmkrc とビルド設定

10.1 latexmkrc の役割

latexmkrc は、latexmk の既定動作をプロジェクトごとにそろえるための設定ファイルです。毎回長いオプションを手で打たなくてよくなり、VSCode から呼ぶ場合も設定を共有しやすくなります。

10.2 LuaLaTeX + biber の設定例

このリポジトリでは、次のような設定を使っています。

```
$latex = 'texfot lualatex -synctex=0 -file-line-error -halt-on-error %0 %S';
$lualatex = 'texfot lualatex -synctex=0 -file-line-error -halt-on-error %0 %S';
$bibtex_use = 2;
$biber = 'texfot biber %0 %S';
$pdf_mode = 4;
$out_dir = '../out';
$aux_dir = '../out/aux';
```

biber を使うために `$bibtex_use = 2` を設定し、`$out_dir` を `out/`、`$aux_dir` を `out/aux/` に分けています。この設定では、最終的な PDF は `out/` に置かれ、中間生成物は `out/aux/` にまともります。補助ファイルを本文と同じ場所へ散らさず、完成物の PDF も中間生成物に埋もれないので、Git 管理でも見通しがよくなります。

10.3 outdir を分ける理由

`out/` 配下で PDF と補助ファイルを分ける理由は、主に次の 3 つです。

- `.aux` や `.log` が `src/` に散らばらない
- 完成した PDF が中間生成物に埋もれない
- Git で差分を見るときにノイズが減る

入門用の単一ファイルでは同じディレクトリに出しても構いませんが、レポートや卒論のようにファイル数が増えると、PDF を `out/`、補助ファイルを `out/aux/` に分ける運用の恩恵が大きくなります。

10.4 VSCode 設定との対応

LaTeX Workshop では、`latexmk` を呼ぶ recipe を用意し、出力先もプロジェクト設定と一致させます。大切なのは、「VSCode だけで動く設定」にせず、ターミナルでも同じ挙動になるように保つことです。

10.5 VSCode settings.json の例

たとえば、次のようにしておくと、LaTeX Workshop から `latexmk` を使いやすくなります。

```
{
  "latex-workshop.latex.autoBuild.run": "never",
  "latex-workshop.latex.outDir": "%DIR%/../out/aux",
  "latex-workshop.latex.recipes": [
    {
      "name": "latexmk (lualatex)",
      "tools": [
        "latexmk_lualatex"
      ]
    }
  ],
  "latex-workshop.latex.tools": [
    {
      "name": "latexmk_lualatex",
      "command": "latexmk",
      "args": [
        "-lualatex",
        "-interaction=nonstopmode",
        "-file-line-error",
        "%DOC%"
      ]
    }
  ]
}
```

11 ファイル構成

11.1 最小構成と実運用構成の違い

最小構成では、`minimal.tex` のように 1 ファイルだけで十分です。一方で、実際のレポートや卒論では、本文、図、参考文献、出力ファイルを分けた方が管理しやすくなります。

11.2 推奨構成

```
project/
├── src/
│   ├── main.tex
│   ├── section/
│   └── sty/
├── fig/
├── bib/
├── out/
│   └── aux/
├── latexmkrc
├── .gitignore
└── .vscode/
```

この構成には、それぞれ理由があります。

- `src/`: 本文ソースを集める。大きくなったら `\input` で章ごとに分割する
- `fig/`: 図ファイルをまとめる。画像素材の差し替え先が明確になる
- `bib/`: `.bib` をまとめる。複数文書で文献データを再利用しやすい
- `out/aux/`: 補助ファイルを隔離する
- `out/`: 完成した PDF をまとめる
- `.vscode/`: エディタ設定を共有する

`\input` による分割は、章や節が増えたときの保守性に効きます。Git でも、本文の変更と図の変更と設定変更が分かれやすくなるため、差分レビューがしやすくなります。

11.3 `.gitignore` の例

生成物を Git 管理に混ぜないために、`.gitignore` を用意しておく安全です。

```
*.aux
*.bbl
*.bcf
*.blg
*.fdb_latexmk
*.fls
*.log
*.out
*.run.xml
*.synctex.gz
*.toc
```

```
out/aux/
```

PDF を成果物として残したいなら `out/` は除外せず、補助ファイルだけが入る `out/aux/` を無視する、という分け方がしやすくなります。

11.4 Makefile の簡単な例

`latexmk` を使うなら必須ではありませんが、コマンドを短くそろえたい場合は `Makefile` を置いても構いません。

```
all:
    latexmk -lualatex src/introduction-latex.tex

minimal:
    latexmk -lualatex src/minimal.tex

clean:
    rm -rf out/aux
```

複数人で作業するときに、ビルド手順を ‘make all’ のような形で共有しやすくなります。

12 よくあるエラー

12.1 閉じ忘れ

初学者が最もよく引っかかるのは、波括弧や数式区切りの閉じ忘れです。

- `{` と `}` の対応が取れていない
- `$` を閉じ忘れている
- `\begin` と `\end` の環境名が一致していない

エラーが出た行だけでなく、その少し前から見るのが基本です。実際の原因は、エラーメッセージ行より上にあることがよくあります。

12.2 全角記号の混入

見た目は似ていても、全角の `(` や `¥` が混ざると、命令や数式として解釈されません。特に日本語 IME を有効にしたままコードを書くと、全角記号が紛れ込みやすくなります。

12.3 package 読み込み順

`hyperref` を前に置きすぎると、後から読む `package` との相性で警告や不具合が出ることがあります。また、`mathtools` は `amsmath` の後に置くのが前提です。「とりあえず動いた」状態でも、読み込み順の意図はコメントで残しておく方が安全です。

12.4 補助ファイルを消すと直ることがある

参照や目次、参考文献まわりが壊れたときは、まず `out/aux/` を消して再ビルドすると直ることがあります。古い `.aux` や `.bcf` が残っていて、現在のソースと整合しなくなるためです。

13 避けるべき書き方

次のような書き方は、入門段階から避けた方がよいです。

- `$$ ... $$`: `equation` や `align` の方が番号付けや拡張に強い
- 手動スペース調整の乱用: 見た目合わせだけの空白は後で崩れやすい
- 過度な `\\`: 段落を書くべき場所まで改行で済ませない
- 位置調整目的だけの空白: 構造ではなく見た目に依存した文書になる
- フォントサイズ直指定の乱用: 局所修正が積み重なると全体設計が崩れる

L^AT_EX らしい運用とは、見た目を力ずくで押し込むことではなく、構造と意味を先に書くことです。その方が、レポートから卒論、さらに論文へと文書規模が大きくなっても破綻しにくくなります。

14 発展トピック

最初の 1 本を書けるようになったら、次のような話題へ進むと実運用に近づきます。

- `siunitx`: 数値と単位の表記をそろえる
- `TikZ`: 簡単な図をソースから描く
- `\newcommand`: よく使う表記をコマンド化する
- 数式スタイルガイド: 添字、関数名、ベクトル表記を文書全体でそろえる
- 卒論・論文向け構成: `\input` や `\include` で章ごとに分割する

ただし、これらは最小構成が安定して通るようになってから追加する方が学習しやすくなります。

15 参考資料

より詳しく学ぶときは、Learn LaTeX 日本語版 [4]、TeX Wiki [5]、jlreq documentation [2]、Overleaf Japanese Guide [1]、そして `hello.tex` の元資料 [6] を参照すると役立ちます。

16 まとめ

この資料で押さえた流れは、次の順です。

1. まず `minimal.tex` で LuaLaTeX + jlreq の最小構成を通す
2. 必要に応じて `package` を追加する
3. 数式では `equation`、`align`、`aligned` を使い分ける
4. 図表や数式番号は `\label` と `\ref`、`\eqref` で参照する
5. 参考文献は `biblatex + biber` で `.bib` に分離する
6. ビルド設定とディレクトリ構成を早めに整える

これらは、短い講義レポートを書くときだけでなく、将来的に卒業論文や論文原稿へ拡張するときの土台になります。最初からすべてを覚える必要はありませんが、「最小構成から 1 機能ずつ増やす」

という進め方は、そのまま実運用でも有効です。

References

- [1] *Japanese - Overleaf, Online LaTeX Editor*. URL: <https://www.overleaf.com/learn/latex/Japanese> (visited on 05/11/2026).
- [2] *jlreq*. URL: <https://tug.org/docs/latex/jlreq/jlreq.html> (visited on 05/11/2026).
- [3] Leslie Lamport. *LaTeX: A Document Preparation System*. 2nd ed. Addison-Wesley, 1994.
- [4] *Learn LaTeX (日本語版)*. URL: <https://www.learnlatex.org/ja/> (visited on 05/11/2026).
- [5] *TeX Wiki*. URL: <https://texwiki.texjp.org/> (visited on 05/11/2026).
- [6] Takayuki YATO. *LaTeX: Which kind of “LaTeX” am I using?* URL: <https://gist.github.com/zr-tex8r/27bc6ddf376d4f716e7276c7dad5ec75> (visited on 05/11/2026).